THE UNIVERSITY OF AZAD JAMMU AND KASHMIR, MUZAFFARABAD

Software Testing

LAB TASK 4



Submitted to: Engr.Fahad Nisar

Submitted by: Faizan Shabbir Abbasi

2024-SET-21

Software Testing Lab

Lab Task 04: Unit Testing: Implement a Taxi-Fare Module and Verify It

1. Objective

Implement six taxi-fare functions strictly from a written list of rules (R1–R6), then verify every rule with Node.js unit tests (node:test) written in the Arrange–Act–Assert structure. The function totalDistance (R6) was developed test-first: its tests were written and run (red) while the function body was still empty, and only then was the function implemented (green).

2. Project Setup

- npm init -y
- npm pkg set type=module
- npm pkg set scripts.test="node --test"
- Folders: src/ (taxi.js) and test/ (taxi.test.js). Tests are run with npm test.

3. Implementation Summary (src/taxi.js)

Each function was written only from its own rule. A ride is an object { distanceKm, ratePerKm }.

Rule	Function	How it was implemented
R1	fareForRide(distanceKm, ratePerKm)	Math.round(distanceKm * ratePerKm).
R2	totalFare(rides)	Loops over the rides and adds fareForRide for each ride (each fare is rounded per R1 before adding). Starts at 0, so an empty list gives 0.
R3	applyPeakSurcharge(amount, percent)	Math.round(amount * (100 + percent) / 100). Multiplying before dividing avoids floating-point noise such as 1.15 being stored inexactly.
R4	waitingCharge(minutes)	Math.max(0, minutes − 5) × 10. The first 5 minutes are free; the max(0, …) keeps waits of 0–5 minutes from going negative.
R5	rideSummary(rides, surchargePercent, minutesWaited)	fare = totalFare(rides); surcharge = applyPeakSurcharge(fare, percent) − fare; waitingCharge from R4; total = fare + surcharge + waitingCharge. Returns { fare, surcharge, waitingCharge, total }.
R6	totalDistance(rides)	Loops over the rides and adds distanceKm. Written test-first (see section 4). Empty list gives 0.

4. Test-First Record for totalDistance (R6)

- Step 1 – src/taxi.js contained R1–R5 implemented and totalDistance as an empty function body.
- Step 2 – The three R6 tests were written in test/taxi.test.js with expected values worked out by hand (20.5, 0 and 10).
- Step 3 – npm test was run: 19 passed and 3 failed. All three failures were the R6 tests, each reporting “undefined !== expected”. This is the red run (Figure 1).
- Step 4 – The minimum code was added to totalDistance: sum distanceKm over all rides, starting from 0.
- Step 5 – npm test was run again: 22 passed and 0 failed. This is the green run (Figure 2). No expected value was changed between the two runs.

5. Notes on How Tests Were Designed

- Every expected value was calculated by hand from the written rule; the hand calculation is shown in the Assert column of the table and in a comment above each assertion in the code.
- Inputs were chosen to avoid ambiguity. The rules say “nearest whole rupee” without stating how an exact .5 is treated, so no test uses an exact .5 result. Fractions such as .4, .6, .85 and .3 are used instead.
- Distances such as 2.5 km were chosen because they are exactly representable in binary floating point, so the sum compares reliably with strictEqual.
- waitingCharge covers all three required boundaries (4, 5 and 6 minutes) plus 0 and a long wait of 15 minutes.
- Tests are grouped with describe (one group per rule) and every test name describes the behaviour being checked.

6. Unit Test Record Table

One row per test (22 tests). “Fig. 1” is the red run (totalDistance not yet implemented); “Fig. 2” is the green run (all tests passing).

#	Function / rule	Test name (exact string)	Arrange	Act	Assert (expected value by hand, assertion used)	Result	Evidence
1	fareForRide – R1	multiplies distance by rate for a whole-number fare	distanceKm = 10, ratePerKm = 25 (simple whole numbers)	fareForRide(10, 25)	Expected 250 (10 × 25 = 250). strictEqual	Pass	Fig. 1 (red run): pass. Fig. 2 (green run): pass.
2	fareForRide – R1	rounds a fare down when the fraction is below .5	distanceKm = 3.7, ratePerKm = 12 (gives a .4 fraction)	fareForRide(3.7, 12)	Expected 44 (3.7 × 12 = 44.4 → 44). strictEqual	Pass	Fig. 1 (red run): pass. Fig. 2 (green run): pass.
3	fareForRide – R1	rounds a fare up when the fraction is above .5	distanceKm = 3.8, ratePerKm = 12 (gives a .6 fraction)	fareForRide(3.8, 12)	Expected 46 (3.8 × 12 = 45.6 → 46). strictEqual	Pass	Fig. 1 (red run): pass. Fig. 2 (green run): pass.
4	fareForRide – R1	returns 0 for a ride of 0 km	distanceKm = 0, ratePerKm = 30 (zero-distance edge case)	fareForRide(0, 30)	Expected 0 (0 × 30 = 0). strictEqual	Pass	Fig. 1 (red run): pass. Fig. 2 (green run): pass.
5	totalFare – R2	adds the fare of every ride in the list	3 rides: 10 km @ 25, 3.8 km @ 12, 8 km @ 30	totalFare(rides)	Expected 536 (250 + 46 + 240). strictEqual	Pass	Fig. 1 (red run): pass. Fig. 2 (green run): pass.
6	totalFare – R2	returns 0 for an empty list of rides	rides = [] (empty list rule)	totalFare([])	Expected 0 (no rides). strictEqual	Pass	Fig. 1 (red run): pass. Fig. 2 (green run): pass.
7	totalFare – R2	rounds each ride (R1) before adding, not the final sum	2 identical rides: 3.7 km @ 12 (each 44.4)	totalFare(rides)	Expected 88 (44 + 44); rounding the raw sum 88.8 would wrongly give 89. strictEqual	Pass	Fig. 1 (red run): pass. Fig. 2 (green run): pass.
8	applyPeakSurcharge – R3	increases a round amount by the given percent	amount = 1000, percent = 20 (easy to compute by hand)	applyPeakSurcharge(1000, 20)	Expected 1200 (1000 + 20% of 1000). strictEqual	Pass	Fig. 1 (red run): pass. Fig. 2 (green run): pass.
9	applyPeakSurcharge – R3	rounds up to the nearest rupee when the result has a high fraction	amount = 99, percent = 15 (result has a .85 fraction)	applyPeakSurcharge(99, 15)	Expected 114 (99 × 1.15 = 113.85 → 114). strictEqual	Pass	Fig. 1 (red run): pass. Fig. 2 (green run): pass.
10	applyPeakSurcharge – R3	rounds down to the nearest rupee when the result has a low fraction	amount = 333, percent = 10 (result has a .3 fraction)	applyPeakSurcharge(333, 10)	Expected 366 (333 × 1.10 = 366.3 → 366). strictEqual	Pass	Fig. 1 (red run): pass. Fig. 2 (green run): pass.
11	applyPeakSurcharge – R3	leaves the amount unchanged when the surcharge is 0 percent	amount = 250, percent = 0 (no-surcharge edge case)	applyPeakSurcharge(250, 0)	Expected 250 (250 + 0%). strictEqual	Pass	Fig. 1 (red run): pass. Fig. 2 (green run): pass.

#	Function / rule	Test name (exact string)	Arrange	Act	Assert (expected value by hand, assertion used)	Result	Evidence
12	waitingCharge – R4	is free at 4 minutes (below the 5-minute limit)	minutes = 4 (boundary: just under the free limit)	waitingCharge(4)	Expected 0 (4 ≤ 5, free). strictEqual	Pass	Fig. 1 (red run): pass. Fig. 2 (green run): pass.
13	waitingCharge – R4	is still free at exactly 5 minutes (boundary)	minutes = 5 (boundary: exactly the free limit)	waitingCharge(5)	Expected 0 (first 5 minutes free). strictEqual	Pass	Fig. 1 (red run): pass. Fig. 2 (green run): pass.
14	waitingCharge – R4	charges Rs 10 at 6 minutes (one minute beyond the free time)	minutes = 6 (boundary: first chargeable minute)	waitingCharge(6)	Expected 10 ((6 − 5) × 10). strictEqual	Pass	Fig. 1 (red run): pass. Fig. 2 (green run): pass.
15	waitingCharge – R4	is free at 0 minutes of waiting	minutes = 0 (no waiting at all)	waitingCharge(0)	Expected 0 (nothing waited). strictEqual	Pass	Fig. 1 (red run): pass. Fig. 2 (green run): pass.
16	waitingCharge – R4	charges Rs 10 for every minute beyond 5 on a long wait	minutes = 15 (well past the free time)	waitingCharge(15)	Expected 100 ((15 − 5) × 10). strictEqual	Pass	Fig. 1 (red run): pass. Fig. 2 (green run): pass.
17	rideSummary – R5	returns fare, surcharge, waitingCharge and total for normal rides	2 rides: 10 km @ 25, 8 km @ 30; surcharge 20%; waited 9 min	rideSummary(rides, 20, 9)	Expected { fare: 490, surcharge: 98, waitingCharge: 40, total: 628 } (fare 250 + 240 = 490; 490 × 1.20 = 588, so surcharge 98; (9 − 5) × 10 = 40; 490 + 98 + 40 = 628). deepStrictEqual	Pass	Fig. 1 (red run): pass. Fig. 2 (green run): pass.
18	rideSummary – R5	returns all zeros for no rides, and free waiting	rides = [], surcharge 10%, waited 3 min (empty edge case)	rideSummary([], 10, 3)	Expected { fare: 0, surcharge: 0, waitingCharge: 0, total: 0 } (0 × 1.10 = 0; 3 ≤ 5 free). deepStrictEqual	Pass	Fig. 1 (red run): pass. Fig. 2 (green run): pass.
19	rideSummary – R5	computes the surcharge as the rounded extra rupees on top of fare	1 ride: 3.8 km @ 12 (fare 46); surcharge 15%; waited 6 min	rideSummary(rides, 15, 6)	Expected { fare: 46, surcharge: 7, waitingCharge: 10, total: 63 } (46 × 1.15 = 52.9 → 53, so surcharge 53 − 46 = 7; (6 − 5) × 10 = 10; 46 + 7 + 10 = 63). deepStrictEqual	Pass	Fig. 1 (red run): pass. Fig. 2 (green run): pass.
20	totalDistance – R6	adds the distanceKm of every ride	3 rides: 10 km, 2.5 km, 8 km (2.5 is exact in binary, so no float noise)	totalDistance(rides)	Expected 20.5 (10 + 2.5 + 8). strictEqual	Fail (red) → Pass (green)	Fig. 1 (red run): FAIL – got undefined. Fig. 2 (green run): pass.
21	totalDistance – R6	returns 0 for an empty list of rides	rides = [] (empty list rule)	totalDistance([])	Expected 0 (no rides). strictEqual	Fail (red) → Pass (green)	Fig. 1 (red run): FAIL – got undefined. Fig. 2 (green run): pass.
22	totalDistance – R6	sums distance only and ignores the rate of each ride	2 rides: 4 km @ 100, 6 km @ 1 (very different rates)	totalDistance(rides)	Expected 10 (4 + 6; rates play no part). strictEqual	Fail (red) → Pass (green)	Fig. 1 (red run): FAIL – got undefined. Fig. 2 (green run): pass.

7. Screenshot 1 – RED run (totalDistance still empty)

npm test with the totalDistance body empty: 19 tests pass, 3 fail (all three are the R6 tests, receiving undefined).



Figure 1 – npm test (red): tests 22, pass 19, fail 3.

8. Screenshot 2 – GREEN run (full suite passing)

npm test after totalDistance was implemented: all 22 tests pass.

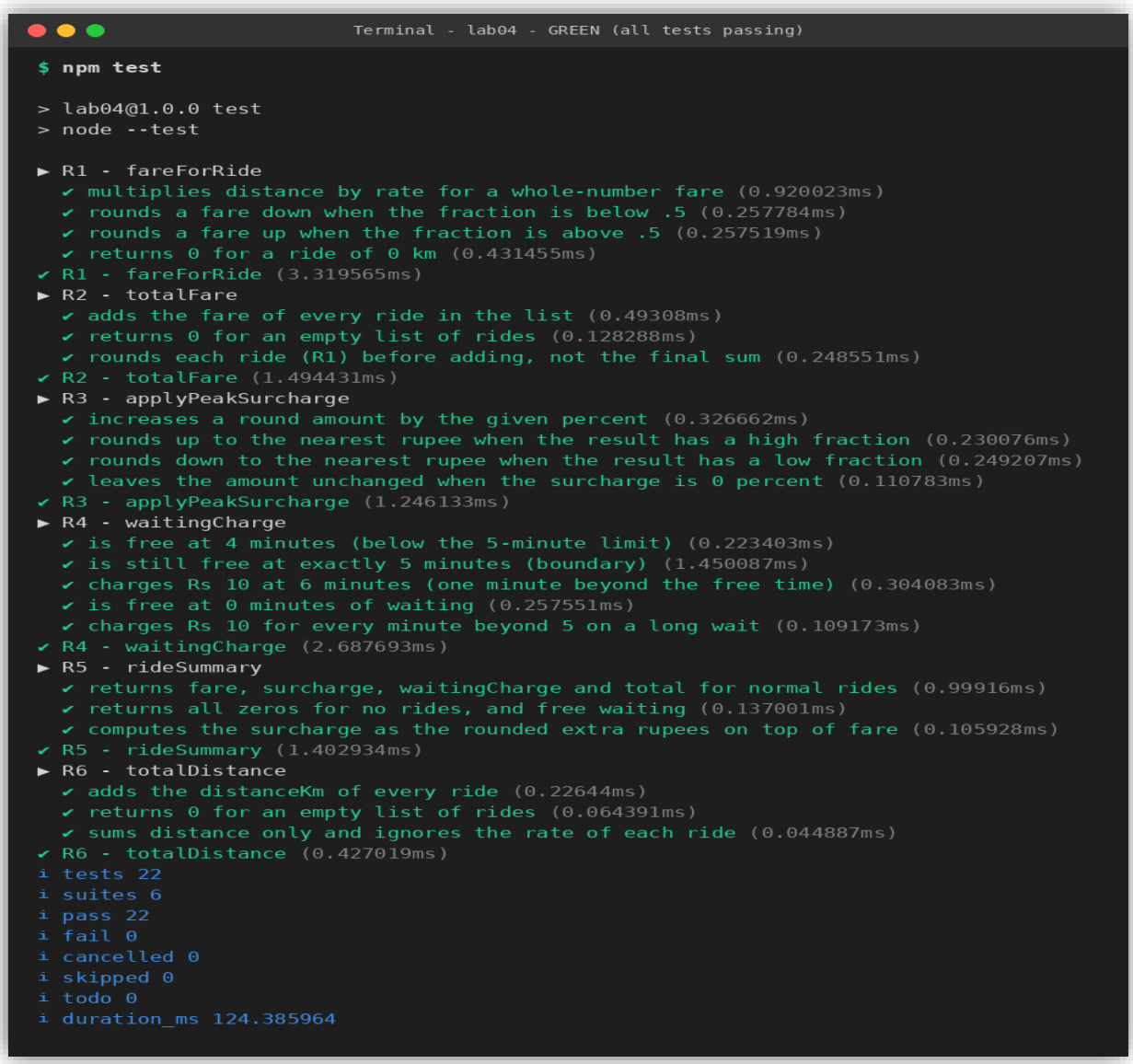


Figure 2 – npm test (green): tests 22, pass 22, fail 0.

9. Execution Summary

Measure	Red run (Fig. 1)	Green run (Fig. 2)
Tests	22	22
Suites (describe groups)	6	6
Passed	19	22
Failed	3 (all R6 totalDistance)	0
Cancelled / skipped / todo	0 / 0 / 0	0 / 0 / 0
Exit status of npm test	1 (failure)	0 (success)

Result: all six functions satisfy their written rules, with 22 passing tests covering R1 (4 tests), R2 (3), R3 (4), R4 (5), R5 (3) and R6 (3).